

GITHUB ACTIONS POUR DEVSECOPS

=====

PIPELINE COMPLET

name: DevSecOps Pipeline

on:

push:

branches: [main]

pull_request:

branches: [main]

jobs:

sast:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Setup Python

uses: actions/setup-python@v4

with:

python-version: '3.11'

- name: SAST - Bandit

run: |

pip install bandit

bandit -r src/ -f json -o bandit-report.json

- name: SAST - SonarCloud

uses: SonarSource/sonarcloud-github-action@master

env:

SONAR_TOKEN: \${{ secrets.SONAR_TOKEN }}

sca:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Check dependencies

run: |

pip install safety

safety check -r requirements.txt --json > safety-report.json

container-scan:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Build Docker image

run: docker build -t myapp:\${{ github.sha }} .

- name: Scan with Trivy

uses: aquasecurity/trivy-action@master

with:

image-ref: 'myapp:\${{ github.sha }}'

format: 'sarif'

output: 'trivy-results.sarif'

dast:

needs: [sast, sca, container-scan]

runs-on: ubuntu-latest

steps:

- name: Deploy app

run: docker-compose up -d

- name: OWASP ZAP Scan

uses: zaproxy/action-baseline@v0.10.0

with:

target: 'http://localhost:8000'

- name: Generate security dashboard

run: |

echo "# Security Report" > security-dashboard.md

echo "## Date: \${date}" >> security-dashboard.md

echo "## Vulnerabilities Found" >> security-dashboard.md

SECURITY BEST PRACTICES

1. Utiliser des tokens avec permissions minimales
2. Ne jamais hardcoder des secrets
3. Scanner les actions tierces avant utilisation
4. Mettre en place des reviews de code obligatoires
5. Configurer des branch protection rules